Trường Đại học Nguyễn Tất Thành
Khoa Công Nghệ Thông Tin

KỸ THUẬT LẬP TRÌNH

TS. Võ Thị Hồng Thắm

CHƯƠNG V.
INTERFACE

Pakage

Package

- Package được sử dụng để **tổ chức mã nguồn theo nhóm có cùng ý nghĩa, cùng mục đích**. Điều này giúp cho việc quản lý mã nguồn được thuận tiện & khoa học. Nhất là trong trường hợp mã nguồn có khối lượng rất lớn các class trong những project thực tế. Có thể thấy khái niệm package tương tự như **khái niệm folder hay directory của hệ điều hành**
- Bên trong package chứa:
  - **class**
  - **Interface**
  - pakage khác – sub package
- Các package, class và các interface có trong 1 package nào đó thường có cùng "*mục đích*"
- Ý nghĩa của modifier "private-public-protected-default" cũng giống như thông thường

Modifier

Type	Cùng class	Cùng package	Trong sub-package	Khác package
private	Có	Không	Không	Không
default	Có	Có	Không	Không
protected	Có	Có	Có	Không
public	Có	Có	Có	Có

Ghi nhớ

- Khi các class được khai báo với các modifier khác nhau thì việc truy cập đến class cũng sẽ khác nhau

default: Được truy cập bởi các class cùng package.

public: Được truy cập bởi tất cả các class ở cùng package hay khác package.

private: Chỉ có thể được truy cập bởi chính class đó.

protected: Được truy cập bởi các class cùng trong package và các class là sub-class của class này.

Sử dụng package

- Cú pháp:
  - import tên_package.tên_class
- Ví dụ:
  - import mypack.MyClass;
  - import mypack.*;
- Ký hiệu *: là import tất cả các class trong package mypack.

Example

*inputStream.java

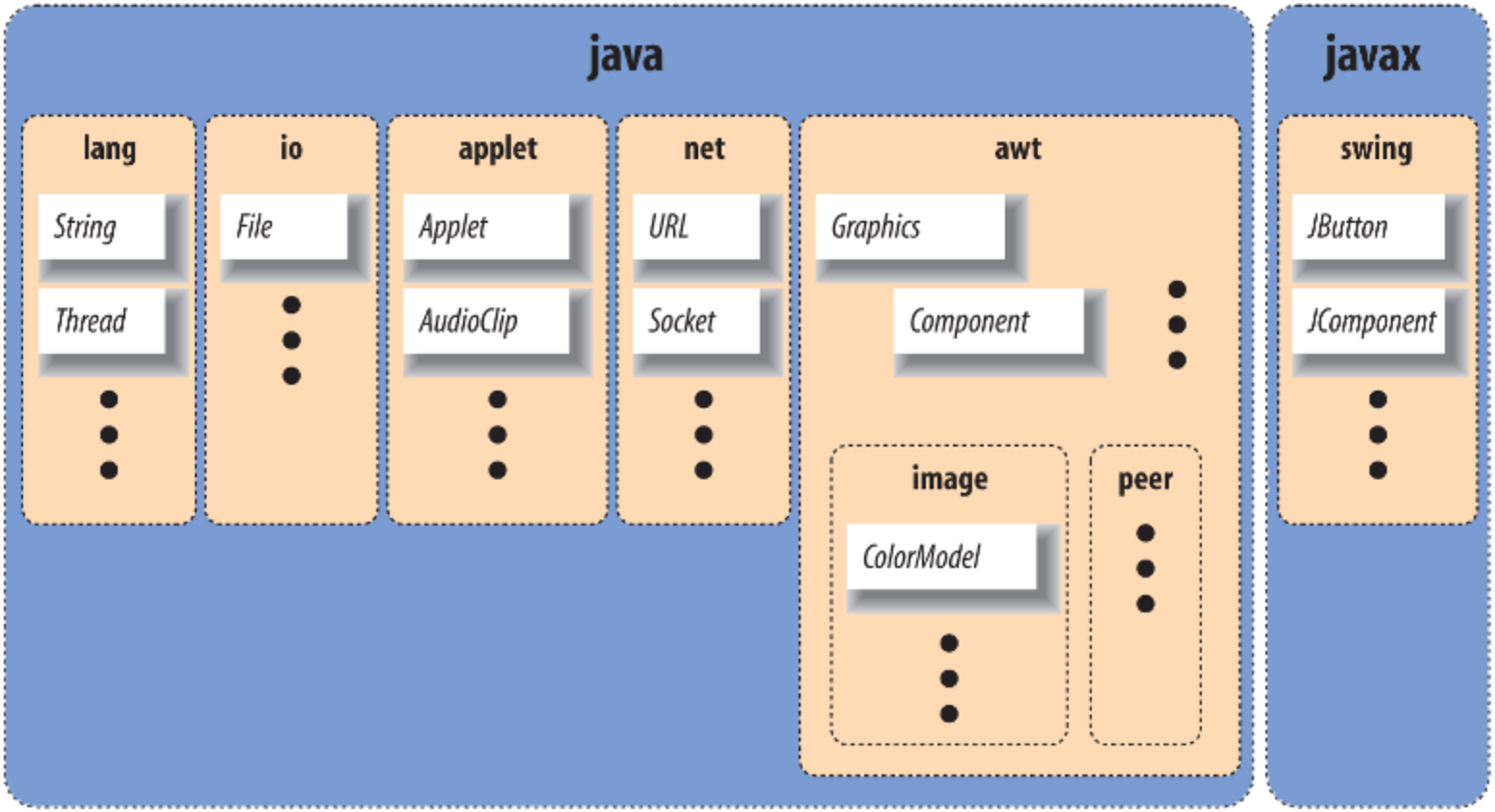
```java
package firstProgram;

import java.io.*;

public class inputStream {

    public static void main(String[] args) {
        try {
            //--- Tạo đối tượng đại diện cho bàn phím
            InputStreamReader reader = new InputStreamReader(System.in);
            //--- Tạo bộ đệm để nhận dữ liệu bàn phím
            BufferedReader buff = new BufferedReader(reader);
            //--- Sử dụng phương thức readLine để đọc 1 dòng dữ liệu
            String s = buff.readLine();
            System.out.print("Xin chào \"" + s +"\", rất vui vì bạn đã đọc được dữ liệu !.");
        } catch (IOException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }

}
```

Khai báo sử dụng package **java.io** của java trong chương trình

Một số package do java cung cấp

Tên Package	Mô tả
java.lang	Chứa các class như Integer, String, System… và được tự động import vào mỗi chương trình Java.
java.util	Các các Java collections như List, Set, Map …
java.io	Chứa các class liên quan đến việc nhập, xuất dữ liệu như File, Reader, Writer…
java.awt và java.swing	Chứa các class liên quan đến việc trình bày giao diện đồ họa và xử lý sự kiện.
. . .	

Common package in java



Ví dụ

Nếu bạn đang phát triển một ứng dụng Java cho việc quản lý thông tin sinh viên, bạn có thể tạo một package tên là "students" để chứa tất cả các lớp và các chức năng liên quan đến sinh viên, chẳng hạn như:

- students.Student: Lớp chứa thông tin của sinh viên.
- students.StudentList: Lớp chứa danh sách sinh viên và các phương thức để thêm, xóa và tìm kiếm các sinh viên.
- students.StudentGUI: Lớp để tạo giao diện cho việc nhập và hiển thị thông tin sinh viên.
- students.StudentDatabase: Lớp chứa các phương thức để kết nối với cơ sở dữ liệu và lưu trữ thông tin sinh viên.

Ví dụ (tt)

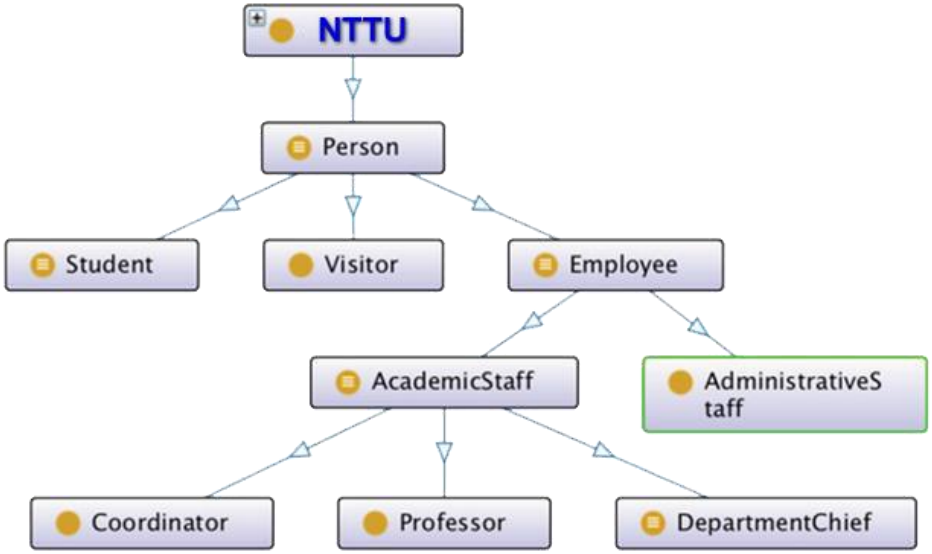
- Dễ dàng quản lý, tìm kiếm và sửa chữa.
- Khi muốn sử dụng các chức năng liên quan đến sinh viên từ một package khác, bạn chỉ cần import package "students" vào file Java của mình.

```
import students.Student;
import students.StudentList;

public class Main {
  public static void main(String[] args) {
    // sử dụng các lớp và phương thức liên quan đến sinh viên
    Student student = new Student("John", "Doe", "12345");
    StudentList list = new StudentList();
    list.addStudent(student);
  }
}
```

interface

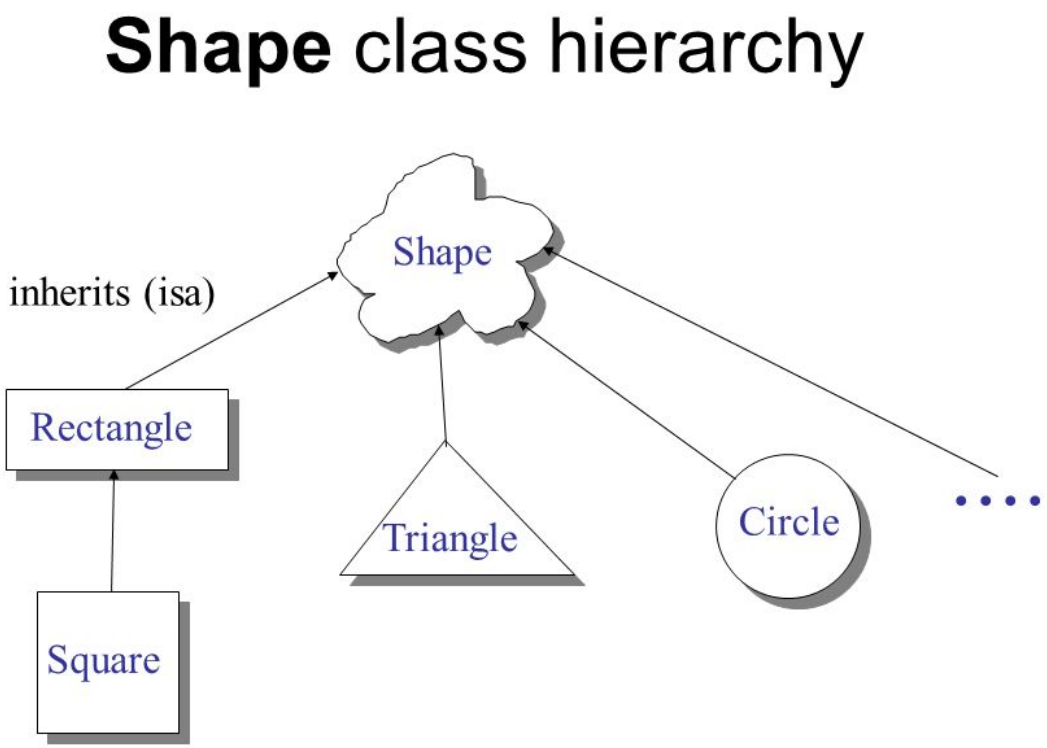
Một chút về abstract class

- Thông thường, trong các "***cấu trúc phả hệ***", các Class ở "**mức cao**" (*Gần gốc*) thường rất "***trừu tượng***", chung chung, …
  - VD: trong cấu trúc NTTU, class **Person**: là trừu tượng.
  - Mặc dù **Person** là rất trừu tượng, nhưng class này lại thể hiện các đặc trưng của hầu hết các sub class là đại diện cụ thể cho 1 loại đối tượng nào đó trong hệ thống chương trình quản lý của NTTU, VD: *name*, *birthDay*, *address*, *phone*, *status*, etc.
  - Như vậy, **Person** nên là một abstract class. Trong trường hợp này, chúng ta không thể tạo ra 1 object của **Person**. Nhưng các lớp con thừa kế từ nó như: Student, Visitor, … thì có thể



Một chút về abstract class

- Ở một tình huống khác, hãy quan sát lớp Shape trong cấu trúc các lớp được thiết kế cho mục tiêu "***đồ họa máy tính***"
  - Shape là một "***lớp trừu tượng***"
  - Bởi nói đến Shape thì người ta có thể tưởng tượng ra hình tròn, tam giác, hình chữ nhật, hình thang, …
  - Tuy nhiên, cho dù là hình dạng nào thì chúng đều có thể có: diện tích, chu vi, cách vẽ, …
  - Như vậy, chúng ta sẽ cần định nghĩa lớp Shape là trừu tương. Lớp này nên có các hàm trừu tượng: dienTich(), chuVi(), veHinh().



Abstract class Shape

```
abstract class Shape {
   public abstract void veHinh();
   // Drawing function must be implemented in each
   // derived class but no default is possible: abstract

   public abstract double dienTich();
   // dienTich function must be implemented in each
   // derived class but no default is possible: abstract

   public abstract double chuVi();
   // chuVi() function must be implemented in each
   // derived class but no default is possible: abstract

   …};

class Square extends Shape {…};
class Circle extends Shape {…};
```

Interface

- Interface là một tập các phương thức mà một class "***<u>buộc phải định nghĩa</u>***" khi triển khai
  - **Interfaces** làm nhiệm vụ khai báo các phương thức mà **class** cần phải có, nhưng không có phần "***thân hàm***" – (*function body*)
  - Khi một **class** implements từ **interface** nào đó thì bắt buộc phải **override** tất cả các phương thức đã được khai báo trong **interface** tương ứng

Interface

- Có thể Interfaces giống như abstract classes, nhưng chúng khác nhau ở chỗ:
  - Một lớp con – subclass chỉ có thể thừa kế bởi ***duy nhất*** một lớp cha – superclass
  - *Tuy nhiên,* ***có thể sử dụng nhiều*** interfaces để làm tiêu chuẩn cho việc định nghĩa một class
  - Một interface không thể có thể hiện cụ thể, tức là *không thể tạo ra đối tượng của interface thông qua toán tử new*

    Comparable list= **new** Comparable(); *//-- Báo lỗi, vì Comparable là một interface*
  - Nhưng bạn lại có thể khai báo 1 đối tượng là interface

    Comparable list; //-- OK
  - Cũng có thể đặt tên cho đối tượng của class đã triển khai từ interface tương ứng:

    Comparable list= **new** MySortClass(); //-- OK, trong trường hợp MySortClass implements từ Comparable
  - Interfaces có thể chứa các khai báo ở dạng phương thức – method hoặc các hằng

```
public interface Rotatable {
    void rotate(double theta);          //--- List reqd methods
    double MAX_ROTATE= 360;
} //--- Implicitly final
  //--- Methods and fields default to be public
```

Interface

- Interfaces cho phép thừa kế, tức là có thể tạo ra 1 interface mới, thừa kế từ 1 interface đã có
- – Scholarship program eligibility
  public interface Eligible {
  boolean IsEligible(double age, double income); }
- – Scholarship program actual selection
  public interface Selected extends Eligible {
  boolean IsSelected(double gpa, double terms); }
- – Our student program could have a scholarship selection class that operates on objects of Student classes that implement one or both of these interfaces
- • Undergrad, Grad, SpecGrad classes from Lecture 12 would all need to implement these methods

Example:: Khai báo interface

```
package mypackage;
interface myinterface{
    void mymethod1();
    void mymethod2();
}
```

Example:: Implement 1 class từ interface

```
package mypackage;
class myclass implements myinterface{

   public void mymethod1(){ //phải là public
      System.out.println("Override my method 1");
   }

   public void mymethod2(){
      System.out.println("Override my method 2");
   }

   void mymethod3(){//không là method trong interface
      System.out.println("My method 3");
   }
}
```

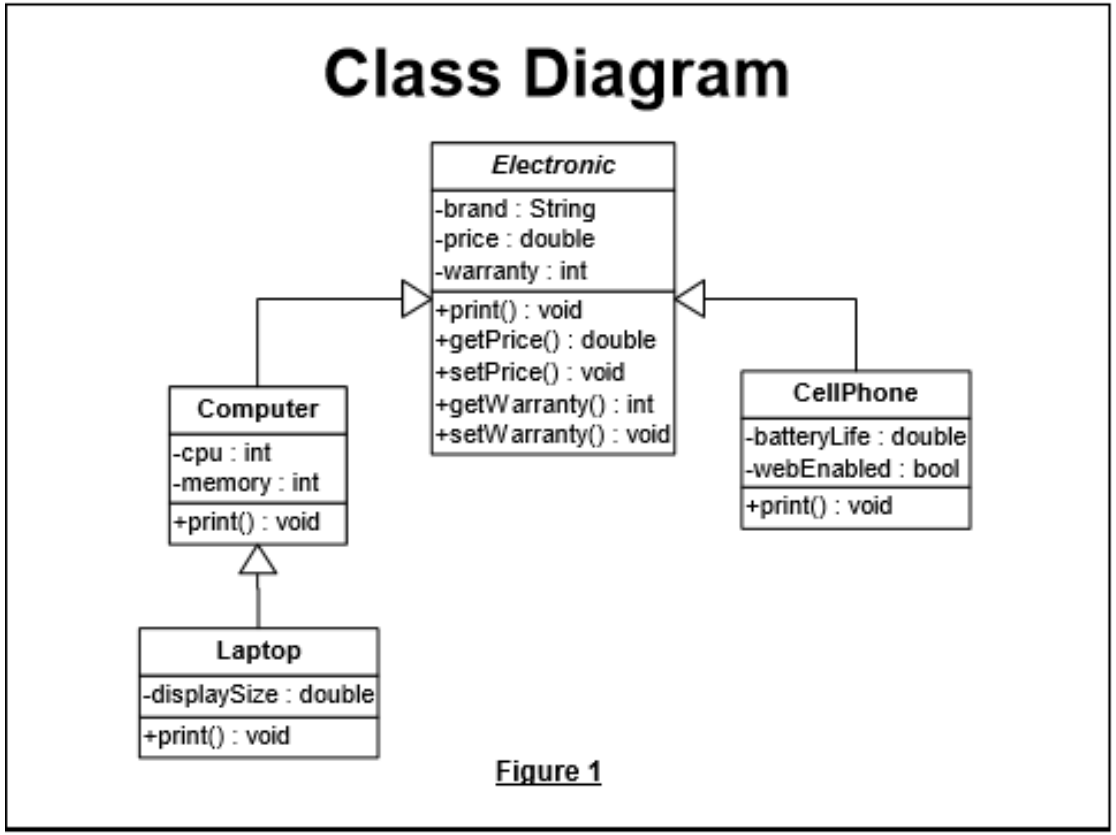
Example:: Sử dụng class đã implement từ myinterface

```
public static void main (String a[]){

            myclass obj1 = new myclass();
            obj1.mymethod1();
            obj1.mymethod2();
            obj1.mymethod3();

            myinterface obj2 = new myclass();
            obj2.mymethod1();
            obj2.mymethod2();
            obj2.mymethod3();// error;
        }
}
```

Example:: Thừa kế interface

```
interface A{
        void meth1();
        void meth2();
}
interface B extends A{
        void meth3();
}
class MyClass implements B{
        public void meth1(){
                System.out.println("Implements method 1");
        }

        public void meth2(){
                System.out.println("Implements method 2");
        }
       public void meth3(){
                System.out.println("Implements method 3");
        }
}
```

Practice


Class Diagram
Electronic
-brand : String
-price : double
-warranty : int
+print() : void
+getPrice() : double
+setPrice() : void
+getWarranty() : int
+setWarranty() : void
Computer
-cpu : int
-memory : int
+print() : void
CellPhone
-batteryLife : double
-webEnabled : bool
+print() : void
Laptop
-displaySize : double
+print() : void
Figure 1

Nhớ gì ?!!!

- ✓ Trong **interface** chỉ bao gồm các ***abstract method*** và các ***biến final***
- ✓ Khi một class ***triển khai*** từ một **interface**, thì nó bắt buộc ***phải cụ thể hóa*** (*override*) tất cả các ***abstract method*** có trong **interface** tương ứng
- ✓ Interface là ***public*** hoặc ***default*** và có thể thừa kế
- ✓ Một **interface** có thể được ***hiện thực*** bởi nhiều class khác nhau, và một class cũng có thể ***triển khai*** nhiều **interface** khác nhau

Bạn đã hiểu

- Interface là gì trong Java?
  - Trong Java, Interface là một tập hợp các phương thức trừu tượng mà các lớp khác có thể triển khai.
  - Interface không thể được khởi tạo như một đối tượng và không chứa bất kỳ thuộc tính nào.
  - Interface chỉ định các phương thức mà các lớp triển khai phải có, nhưng không định nghĩa cách triển khai chúng. Các lớp triển khai Interface phải thực hiện tất cả các phương thức được chỉ định trong Interface.

- Interface có thể chứa các thành phần nào?

Phân biệt interface và abstract class trong java?

Định nghĩa:

- Interface là một bản thiết kế cho các phương thức mà các lớp khác có thể triển khai. *Interface chỉ định tên, kiểu và số lượng tham số* của các phương thức, nhưng không cung cấp bất kỳ định nghĩa phương thức nào.
- Abstract class là một lớp trừu tượng mà không thể được khởi tạo. *Abstract class có thể chứa phương thức trừu tượng hoặc phương thức được triển khai, tuy nhiên nó không thể cung cấp một định nghĩa hoàn chỉnh* cho tất cả các phương thức.

Phân biệt interface và abstract class trong java?

Đặc điểm:

- Interface ***không*** chứa bất kỳ phần thân (implementations) của phương thức nào. Tất cả các phương thức đều phải được triển khai bởi các lớp khác.
- Abstract class ***có thể chứa các phần thân*** (implementations) của phương thức, tuy nhiên, nó không cung cấp đầy đủ định nghĩa cho tất cả các phương thức.

Phân biệt interface và abstract class trong java?

Số lượng:

- Một lớp có thể triển khai nhiều interface.
- Một lớp chỉ có thể kế thừa một abstract class.

Phân biệt interface và abstract class trong java?

Dưới đây là một số trường hợp mà bạn nên sử dụng interface và abstract class:

- Sử dụng interface khi bạn muốn định nghĩa một loạt các phương thức mà các lớp khác có thể triển khai.
- Sử dụng abstract class khi bạn muốn xác định một số phần của một lớp, nhưng bạn muốn các lớp con phải triển khai các phần còn lại.
- Sử dụng abstract class khi bạn muốn chia sẻ code giữa các lớp con.

Bài tập

- Khai báo một interface Animal với hai phương thức di chuyển và ăn. Sau đó, chúng ta triển khai interface này trong lớp Dog bằng cách định nghĩa lại hai phương thức của interface. Cuối cùng, chúng ta tạo một đối tượng Dog và gọi hai phương thức của interface Animal thông qua đối tượng Dog. Khi chạy chương trình, kết quả sẽ hiển thị như sau:

```
Chó di chuyển bằng cách chạy.
Chó ăn thức ăn cho chó.
```

Trường Đại học Nguyễn Tất Thành
Khoa Công Nghệ Thông Tin

-Hết chương 4-